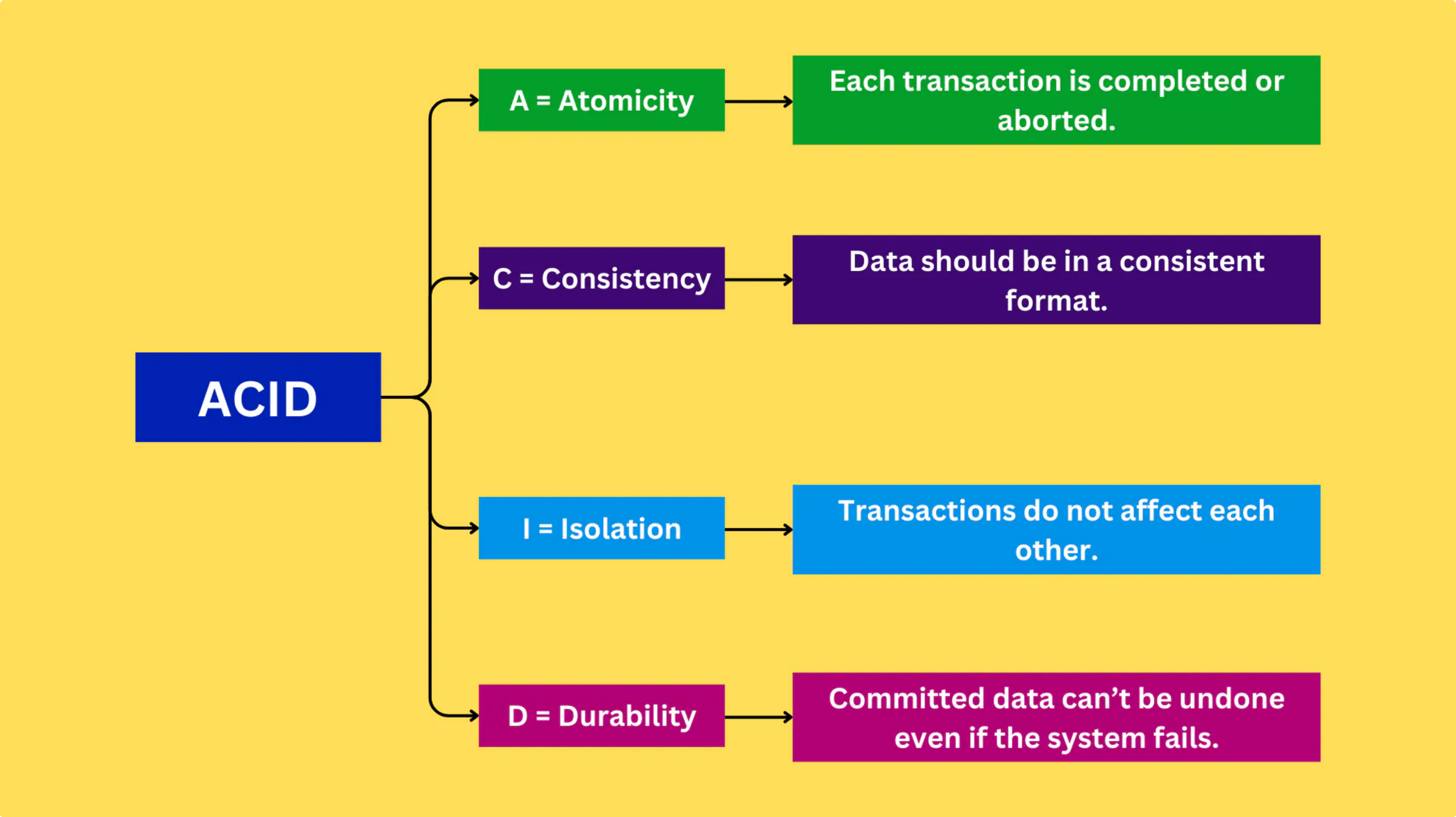
ACID vs BASE Properties in Databases

ACID and BASE are two sets of properties that represent different approaches to handling transactions in database systems. They reflect trade-offs between consistency, availability, and partition tolerance, especially in distributed databases.

ACID Properties

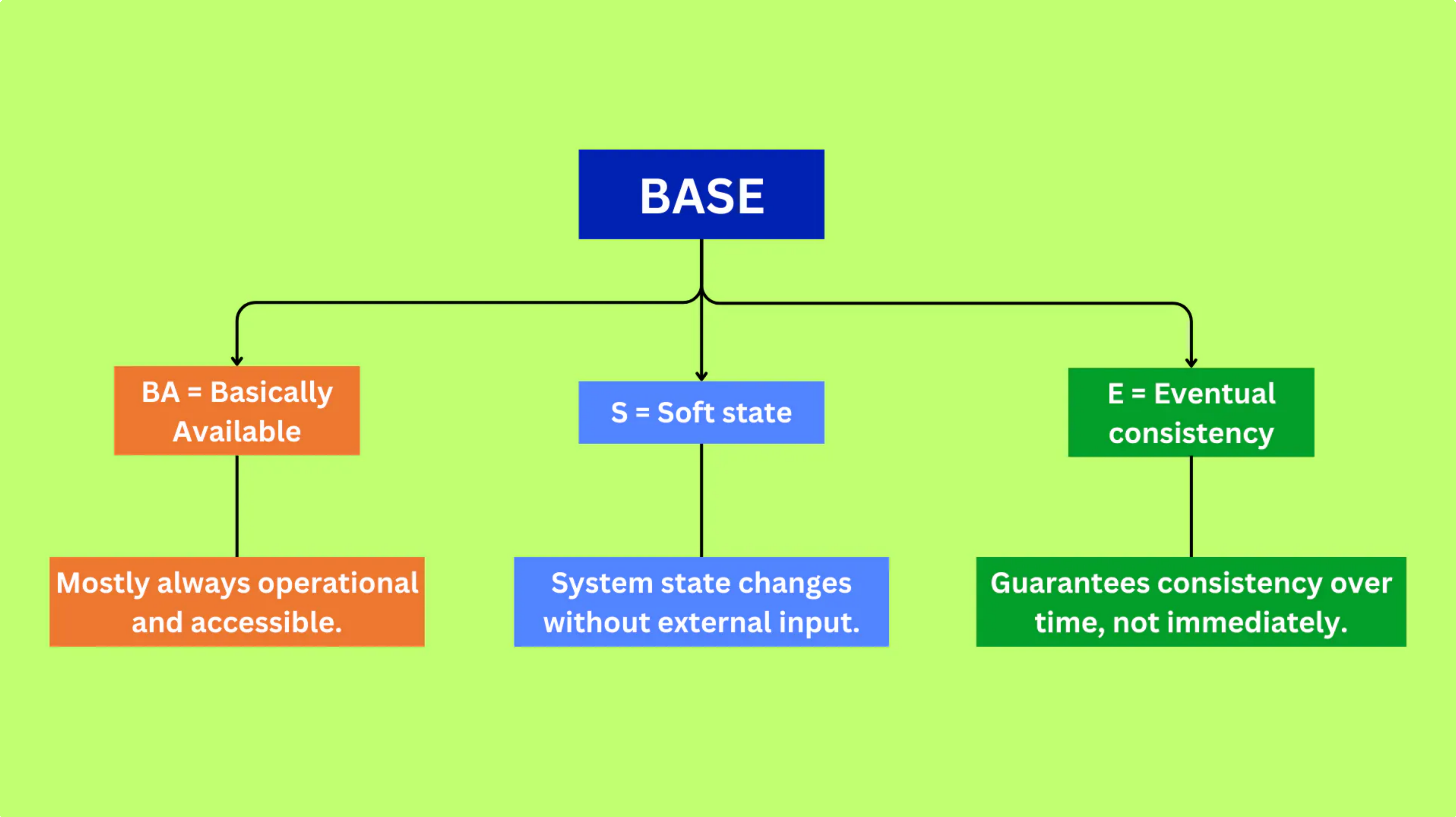
- **Definition:** ACID stands for Atomicity, Consistency, Isolation, and Durability. It's a set of properties that guarantee reliable processing of database transactions.
- **Components:**
  - **Atomicity:** Ensures that a transaction is either fully completed or not executed at all.
  - **Consistency:** Guarantees that a transaction brings the database from one valid state to another.
  - **Isolation:** Ensures that concurrent transactions do not interfere with each other.
  - **Durability:** Once a transaction is committed, it remains so, even in the event of a system failure.
- **Example:** Consider a bank transfer from one account to another. The transfer operation (debit from one account and credit to another) must be atomic, maintain the consistency of total funds, be isolated from other transactions, and changes must be permanent.
- **Use Cases:** Ideal for systems requiring high reliability and data integrity, like banking or financial systems.


ACID
A = Atomicity
Each transaction is completed or aborted.
C = Consistency
Data should be in a consistent format.
I = Isolation
Transactions do not affect each other.
D = Durability
Committed data can't be undone even if the system fails.

ACID

BASE Properties

- **Definition:** BASE stands for Basically Available, Soft state, and Eventual consistency. It's an alternative to ACID in distributed systems, favoring availability over consistency.
- **Components:**
  - **Basically Available:** Indicates that the system is available most of the time.
  - **Soft State:** The state of the system may change over time, even without input.
  - **Eventual Consistency:** The system will eventually become consistent, given enough time.
- **Example:** A social media platform using a BASE model may show different users different counts of likes on a post for a short period but eventually, all users will see the correct count.
- **Use Cases:** Suitable for distributed systems where availability and partition tolerance are more critical than immediate consistency, like social networks or e-commerce product catalogs.


BASE
BA = Basically Available
Mostly always operational and accessible.
S = Soft state
System state changes without external input.
E = Eventual consistency
Guarantees consistency over time, not immediately.

BASE

Key Differences

- **Consistency and Availability:** ACID prioritizes consistency and reliability of each transaction, while BASE prioritizes system availability and partition tolerance, allowing for some level of data inconsistency.
- **System Design:** ACID is generally used in traditional relational databases, while BASE is often associated with NoSQL and distributed databases.
- **Use Case Alignment:** ACID is well-suited for applications requiring strong data integrity, whereas BASE is better for large-scale applications needing high availability and scalability.

Conclusion

ACID and BASE represent different approaches to dealing with data in database systems, each suited to different types of applications. ACID is critical for systems where transactions must be reliable and consistent, while BASE is beneficial in environments where high availability and scalability are necessary, and some degree of data inconsistency is acceptable.

← Previous
Latency Vs Throughput
Next →
Read-Through Vs Write-Through Cache
☐ Mark as Completed